

PORTADA

Análisis y Diseño de Sistemas II

Semana 15 — Código Limpio II: Refactorización y Code Smells

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**Cambiar cómo se ve el código,
sin cambiar lo que hace**

TRANSICIÓN

De nombrar y estructurar bien, a corregir lo que ya existe

El martes se vio la primera mitad de código limpio: nombres significativos y funciones pequeñas que hacen una sola cosa. Eso ayuda a escribir código limpio **nuevo**. Pero la mayor parte del trabajo real no es escribir código desde cero — es modificar código que **ya existe**, y que casi nunca está tan limpio como debería.

Hoy se cierra código limpio con **refactorización**: cómo mejorar la estructura interna de código existente, el catálogo de **code smells** de Martin Fowler que ayuda a reconocer cuándo hace falta, y su conexión directa con los antipatrones ya vistos en la Unidad 5.

CONCEPTO

Qué es refactorizar

Refactorizar (refactoring) es el proceso de cambiar la **estructura interna** de un fragmento de código sin alterar su **comportamiento externo observable**. El término y su catálogo sistemático de técnicas los formalizó **Martin Fowler** en su libro *Refactoring: Improving the Design of Existing Code* (1999).

El punto clave de la definición es "sin alterar el comportamiento externo": el software, desde el punto de vista de quien lo usa, se comporta exactamente igual antes y después. Lo único que cambia es qué tan fácil es de leer, entender y modificar por dentro — es, literalmente, la forma de **pagar el capital de la deuda técnica** vista al inicio de la semana.

CONCEPTO

Refactorizar solo es seguro con pruebas

Fowler es enfático en un requisito previo: refactorizar **sin una red de pruebas automatizadas** que confirmen que el comportamiento no cambió es apostar a ciegas. Sin pruebas, cada refactorización es un salto de fe — se puede introducir un error sin que nadie lo note hasta producción.

Por eso el proceso disciplinado de refactorización sigue un ciclo corto y repetido: **ejecutar las pruebas, hacer un cambio pequeño, ejecutar las pruebas de nuevo**. Si algo falla, se revierte el último cambio pequeño — nunca se avanza sobre una base que ya se sabe rota. Este es el puente directo hacia las pruebas unitarias que se ven la próxima semana.

CONCEPTO

Cuándo refactorizar

- **La regla del Boy Scout:** principio popularizado también por Robert C. Martin — "deja el código un poco más limpio de lo que lo encontraste". No es necesario limpiar todo un módulo de una vez; basta con mejorar algo pequeño cada vez que se pasa por ahí.
- **Antes de agregar una funcionalidad:** si el diseño actual dificulta agregar lo nuevo, primero se refactoriza para que agregarlo sea fácil, y luego se agrega — no al revés.
- **Durante el code review:** cuando un compañero señala una oportunidad de mejora que no cambia el comportamiento, es el momento natural de aplicarla.
- **Al corregir un bug:** mientras se investiga la causa, suele encontrarse la zona de código que más lo merece.



CONCEPTO

Qué es un code smell

Un **code smell** (mal olor de código) es una señal superficial en el código que, sin ser un error en sí mismo, sugiere que existe un problema de diseño más profundo debajo. El término también lo popularizó Fowler en *Refactoring*: el código "funciona", pero algo en cómo está escrito indica que conviene revisarlo.

La relación con lo que se vio en la Unidad 5 es directa: los **antipatrones** (God Object, Spaghetti Code) son el resultado consolidado y con nombre propio de acumular muchos code smells sin corregirlos — un code smell es la señal temprana; un antipatrón es lo que queda cuando esa señal se ignoró durante mucho tiempo.

CONCEPTO

Catálogo de code smells (I)

Code smell	Síntoma
Long Method	Una función que creció tanto que ya no se entiende de un vistazo — el opuesto directo de "funciones pequeñas" visto el martes.
Large Class	Una clase que intenta hacer demasiado — la versión de code smell de lo que en la Unidad 5 se llamó God Object.
Duplicated Code	El mismo fragmento de lógica repetido en varios lugares — la versión de code smell de Copy-Paste Programming.
Long Parameter List	Una función con demasiados argumentos, difícil de llamar y de recordar en qué orden van.

CONCEPTO

Catálogo de code smells (II)

Code smell	Síntoma
Feature Envy	Un método que usa más los datos de otra clase que los propios — señal de que está en la clase equivocada (recuerden Information Expert, semana 10).
Data Clumps	El mismo grupo de datos (ej. día, mes, año) viajando siempre juntos como parámetros sueltos, en vez de agruparse en una clase propia.
Primitive Obsession	Usar tipos primitivos (<code>string</code> , <code>int</code>) para representar conceptos del dominio (email, dinero, rango de fechas) en vez de crear un tipo propio.
Switch Statements	<code>switch</code> o cadenas de <code>if/elif</code> repetidas en varios lugares del código según el mismo tipo — la versión de code smell de Spaghetti Code.

CONTRA EJEMPLO

Primitive Obsession y Data Clumps juntos

```
// Antes: primitivos sueltos que siempre viajan juntos
def crear_reserva(dia, mes, anio, email_cliente):
    if "@" not in email_cliente:
        raise ValueError("email invalido")
    ...

// Después: dos tipos propios del dominio
def crear_reserva(fecha: Fecha, email: Email):
    ... # Fecha y Email ya se validan a sí mismos al construirse
```

`dia`, `mes`, `anio` es un **Data Clump** — siempre viajan juntos y deberían ser una sola clase `Fecha`. Usar un `string` plano para el correo es **Primitive Obsession** — un tipo `Email` propio puede validarse a sí mismo una sola vez, en vez de repetir la validación en cada función que lo recibe.

CONCEPTO

Técnicas básicas de refactorización

Fowler documenta docenas de refactorizaciones con nombre propio y pasos mecánicos. Las cuatro más frecuentes en el día a día:

- **Extract Method:** tomar un bloque de código dentro de una función larga y convertirlo en una función propia con un nombre que explique qué hace — la corrección directa de Long Method.
- **Rename:** cambiar el nombre de una variable, función o clase para que revele mejor su intención, sin tocar su lógica.
- **Extract Class:** mover un grupo de campos y métodos relacionados de una clase sobrecargada a una clase nueva — la corrección directa de Large Class.
- **Replace Conditional with Polymorphism:** reemplazar un `switch` o cadena de `if` que decide comportamiento según un tipo, por una jerarquía de clases polimórfica — el mismo principio que resuelve Switch Statements y Spaghetti Code, y que ya vieron con Strategy y State en la Unidad 5.

TENSIÓN CONCEPTUAL · CIERRE DEL ARCO

Code smells, antipatrones y principios: la misma idea, tres niveles

Con esto se cierra el hilo que conecta las últimas tres semanas del curso: un **principio** no aplicado (SOLID, GRASP) se manifiesta primero como un **code smell** local y pequeño; si se ignora durante suficiente tiempo, se consolida como un **antipatrón** con nombre propio a nivel de todo el sistema.

Code smell (hoy)	Antipatrón (semana 14)	Principio que lo evita
Large Class	God Object	SRP / High Cohesion
Duplicated Code	Copy-Paste Programming	Extract Method/Class
Switch Statements	Spaghetti Code	Strategy / State

INTERACCIÓN

¿Qué code smell y qué refactorización aplicarían?

En el chat, para cada caso indiquen el code smell presente y qué técnica de refactorización lo corrige:

1. Un método `calcularEnvio(peso, destino)` que consulta y usa casi exclusivamente los atributos de la clase `Destino`, y casi nada de su propia clase.
2. Una función de 80 líneas que valida, calcula, guarda y notifica en un solo bloque continuo.
3. Un parámetro `tipoDocumento: string` que recibe literalmente `"CC"`, `"TI"` o `"CE"`, y un `if/elif` distinto por cada valor en cuatro archivos distintos del proyecto.

5 minutos · respondan en el chat

SÍNTESIS

Ideas clave de la sesión

1. Refactorizar es cambiar la estructura interna del código **sin alterar su comportamiento externo**, y solo es seguro con pruebas automatizadas.
2. Se refactoriza con la regla del Boy Scout, antes de agregar funcionalidad, y durante el code review.
3. Un code smell es una señal temprana de un problema de diseño; un antipatrón es lo que queda cuando esa señal se ignora por mucho tiempo.
4. Extract Method, Rename, Extract Class y Replace Conditional with Polymorphism son las técnicas básicas para corregirlos.
5. Con nombres, funciones, comentarios, refactorización y code smells se cierra código limpio — falta la herramienta que hace segura toda refactorización: **pruebas unitarias**.



CIERRE

Antes de la próxima semana

Refactorizar sin pruebas es peligroso. Con esto se cierra la primera semana de la Unidad 6 — deuda técnica y código limpio.

La semana 16, que cierra la unidad, se dedica por completo a **pruebas unitarias de software**: fundamentos y frameworks el martes, TDD y buenas prácticas el miércoles, y un laboratorio práctico sobre el proyecto el viernes.

